NAME: ALLAN NJUGUNA NGUGI

EMAIL: ngugiallan05@gmail.com

ADMISSION NUMBER: CS-SA12-26007

Table of Contents

Link to Module Completion 3
Introduction 3
Web Application Layout 3
Front End vs. Back End 3
HTML 3
Cascading Style Sheets (CSS) 4
Javascript 5
Sensitive Data Exposure 5
HTML Injection 8
Cross-Site Scripting (XSS) 11
Cross-Site-Request-Forgery (CSRF) 13
Back End Servers 13
Web Servers 15
Databases 15
Development Frameworks and APIs 16
Common Web Vulnerabilities 18
Public Vulnerabilities 19
Module Completion 22
Conclusion 22

Link to Module Completion

https://academy.hackthebox.com/achievement/2384006/75

Introduction

At the beginning of this module, I outlined the concepts of the websites and the web applications while putting side by side their pros and cons, respectively. After that, I went through different web application deployments ranging from free to commercial. Finally, I ended the introduction period with a study of web applications' security threats and the regular attacks' methods.

Web Application Layout

I discovered that there is not a single pair of web applications that could be characterized as exactly the same; the diversity is infinite as the businesses create them for such a large number of purposes and different kinds of users. In addition to that, I comprehended that the back-end infrastructure could be the most complicated part of a web app that can be arranged and set up in a myriad of different ways. This awareness helped me to get the importance of being aware of what is going on behind the scenes—from the total structure and separate parts to the integration and the whole thing being matched up with the distinct company setup.

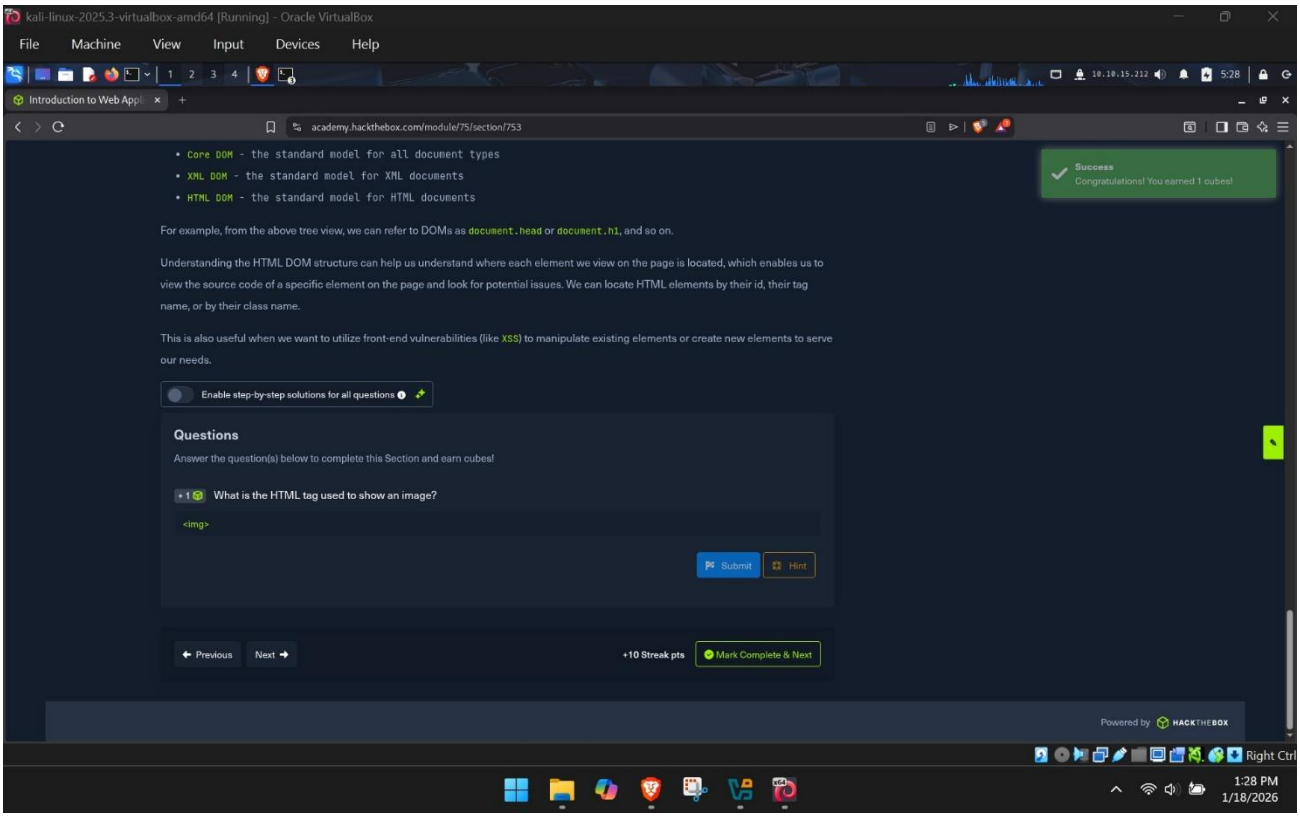
Front End vs. Back End

I came across the fact that "full stack" development consists of both front end and back end but they are separate areas. I found out that front end is concerned with the client-side components that a user sees in their browser while the back end is like the engine, operating on a server, processing all the fundamental logic and features that support the application’s operation.

HTML

After reading through this section, I came to know that HTML forms the basis of any website, it’s the main language which arranges and controls everything from the titles and forms to the pictures. I also got to know that it is the web browser’s task to decipher this markup language, thereby turning the fundamental elements into the very interface that we see and interact with.

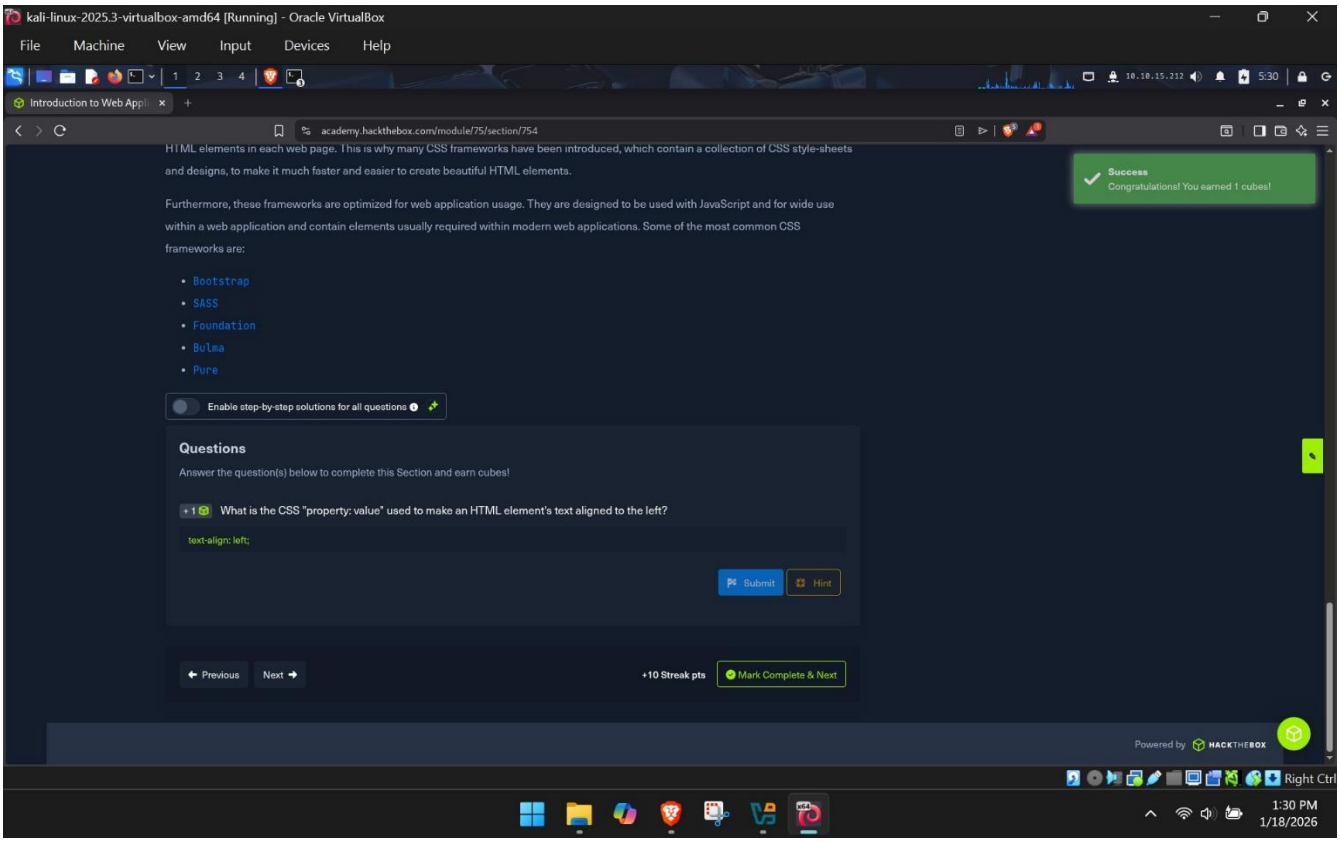
The question is this section is sown below:



Cascading Style Sheets (CSS)

CSS is the system that handles the presentation of HTML files. I found out that with the new releases of CSS, more and more sophisticated formatting options are brought in, and thus the browsers have to keep on updating themselves to be able to showcase these new styles correctly.

The question in this section is shown below.



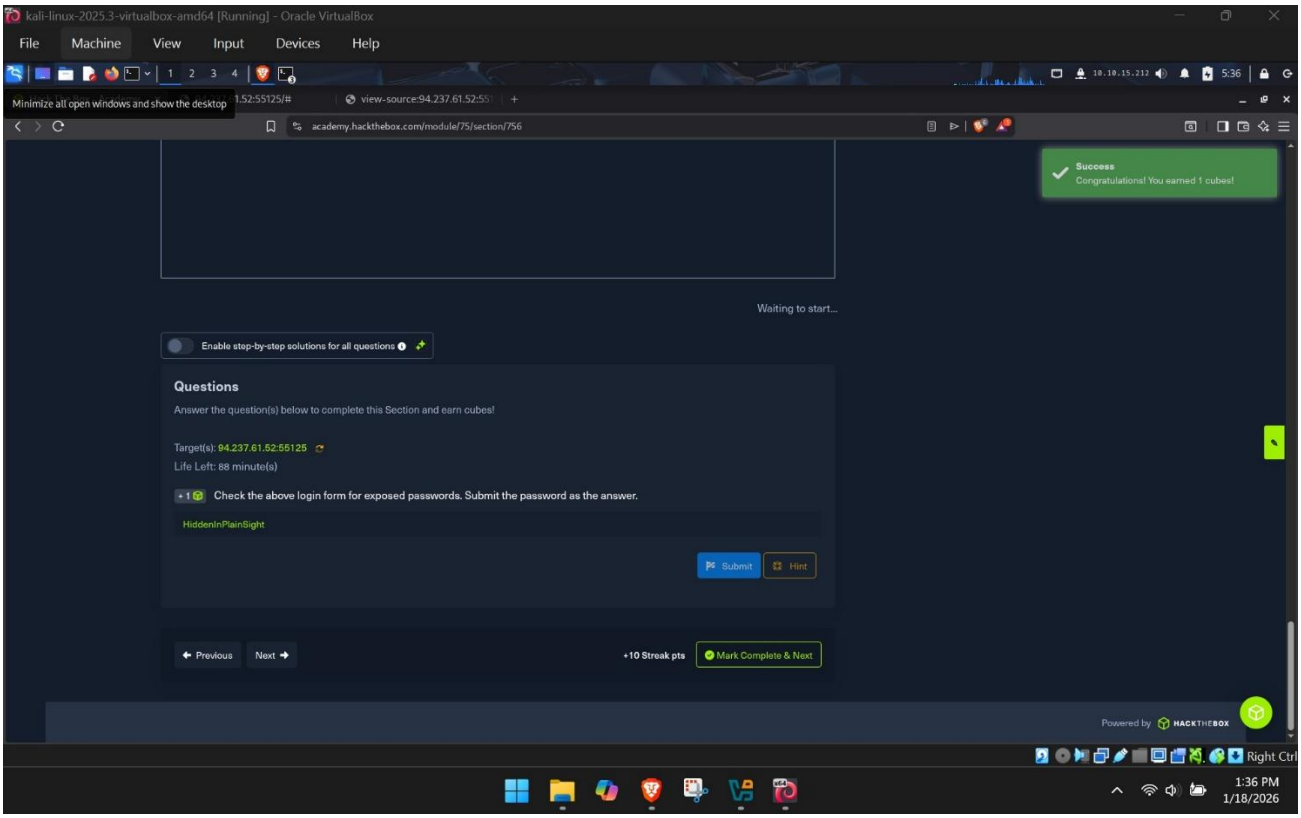
Javascript

JavaScript is a powerful language that is used everywhere in web and mobile development, especially for dynamic page generation. I was aware of the fact that HTML and CSS take care of the aesthetics but it was JavaScript that was responsible for the actual interaction; if not for it the sites would appear rather dull. On the other hand, I learned that even though it's mostly seen as a front-end technology, it can still provide support for the whole back end through technologies like NodeJS.

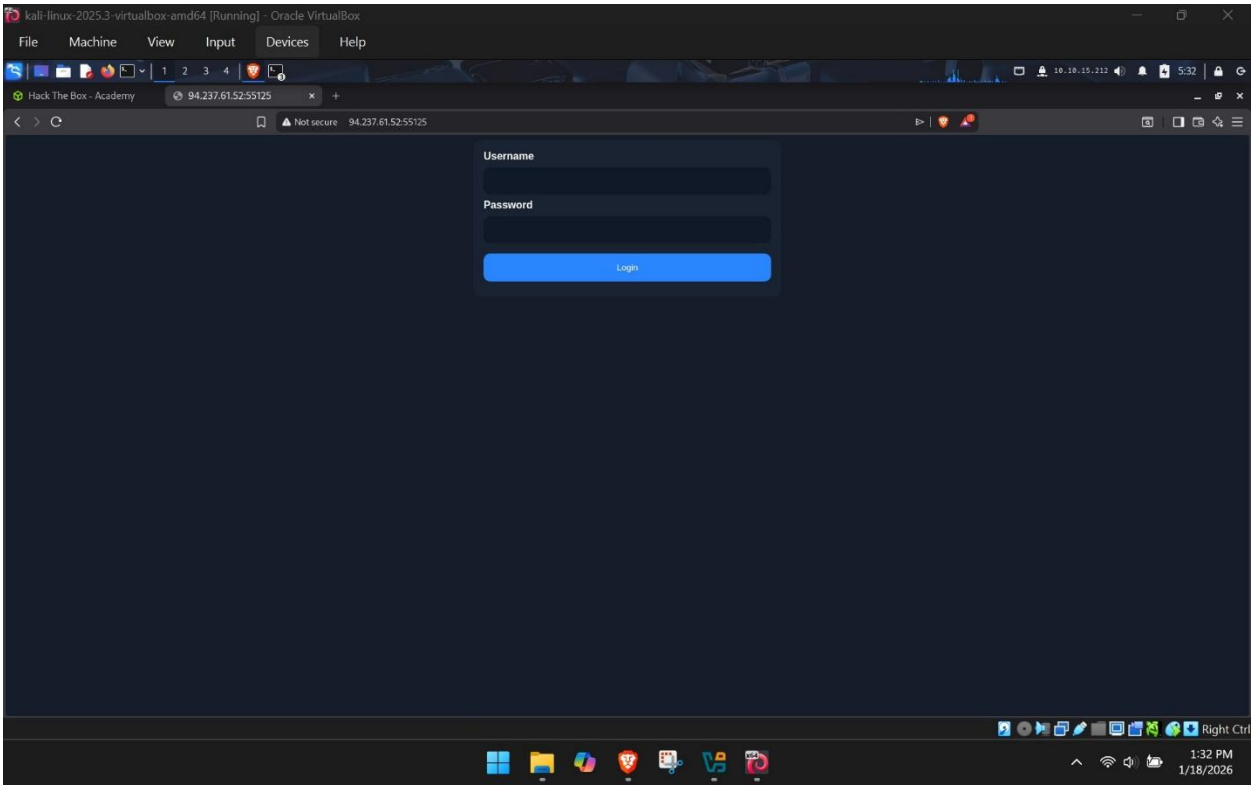
Sensitive Data Exposure

What I found out is that the exposure of sensitive data occurs when the personal data is kept in clear text within the front-end source code of the website. I also realized that even though the back-end code is not visible and stays on the server, the HTML code can be easily seen by anyone who uses right-click or shortcut keys like Ctrl + U. I also learned that even if a developer tries to prevent right-clicking on the page, there are still ways to uncover all the hidden parts of the code using tools like Burp Suite or browser commands that are quite simple.

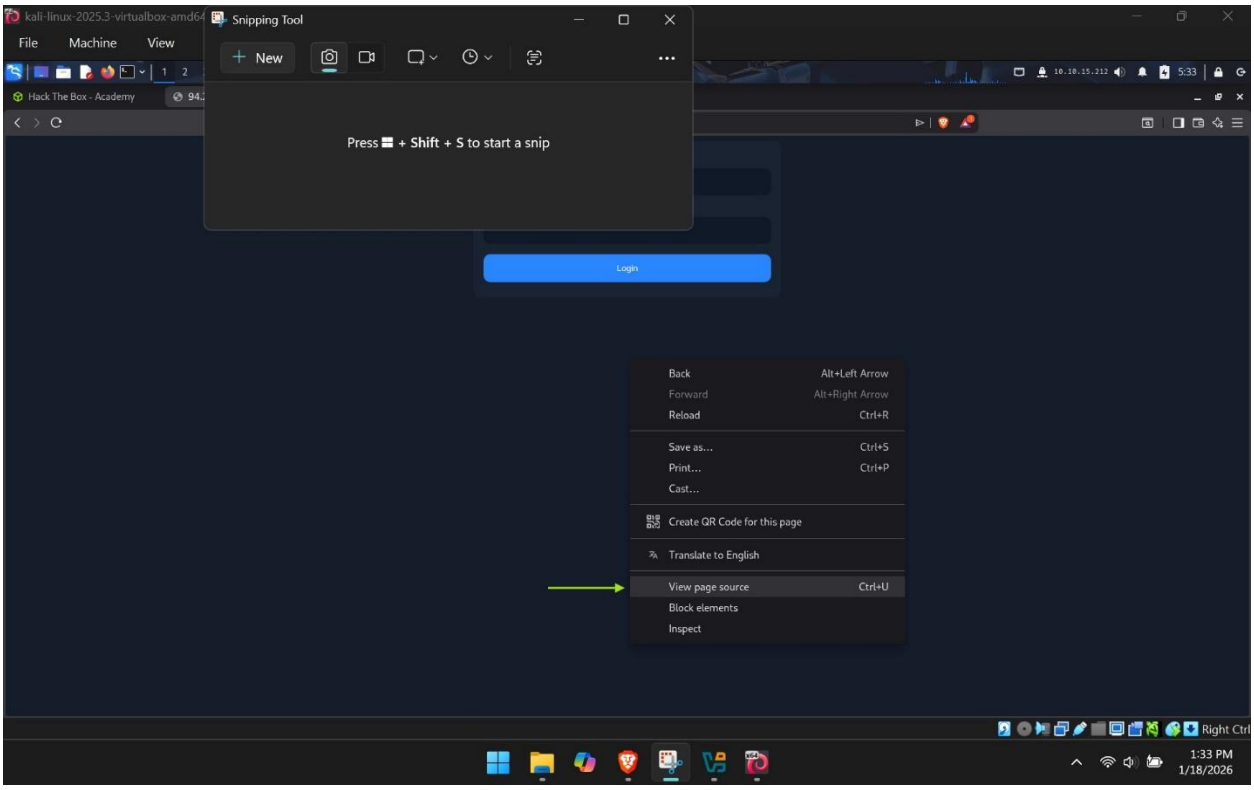
Question



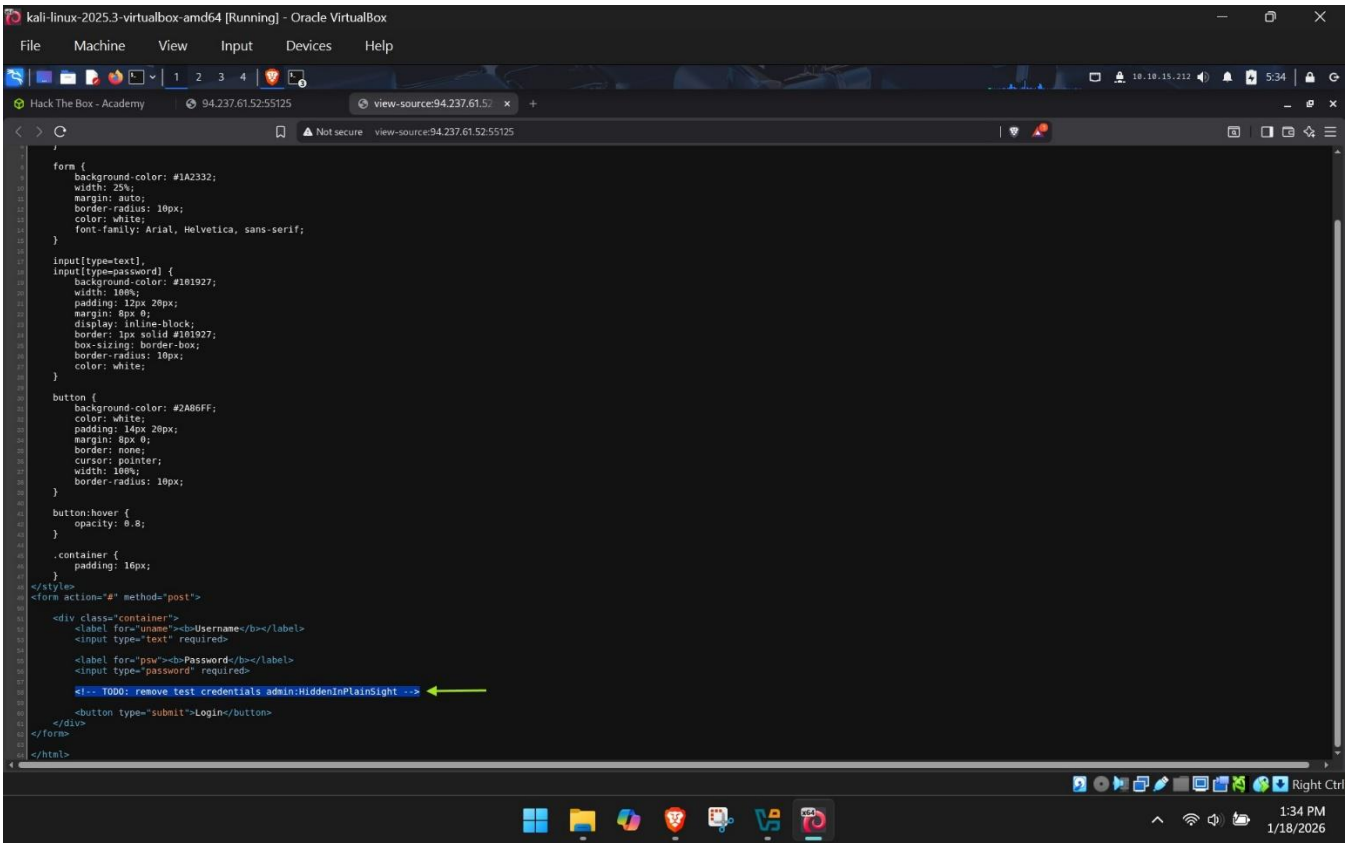
Once I loaded the target in my browser, I was able to load the login page of the target as shown below:



I then right-clicked to find the option of viewing the page source, as shown below:



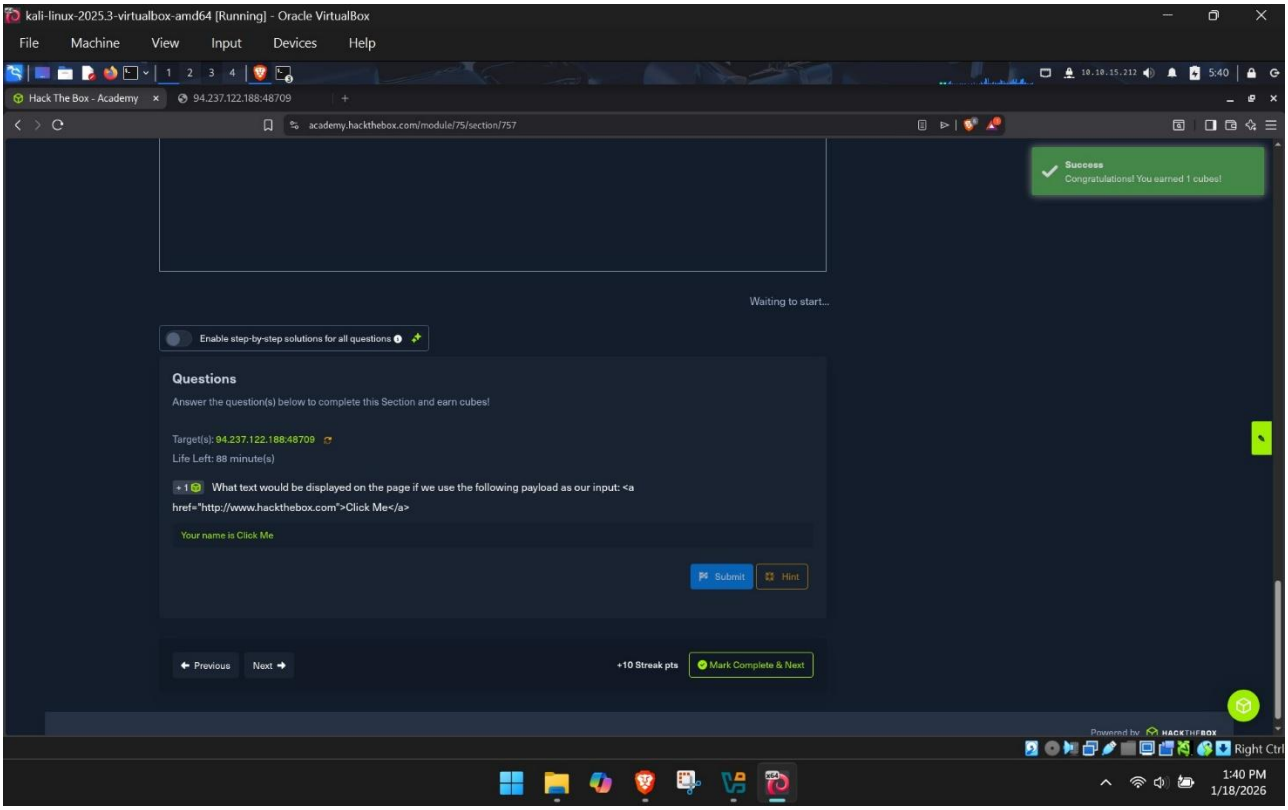
After going through the page source, I was able to find the credentials as ‘admin’ being the username and ‘HiddenInPlainSight’ being the password, as shown in the image below:

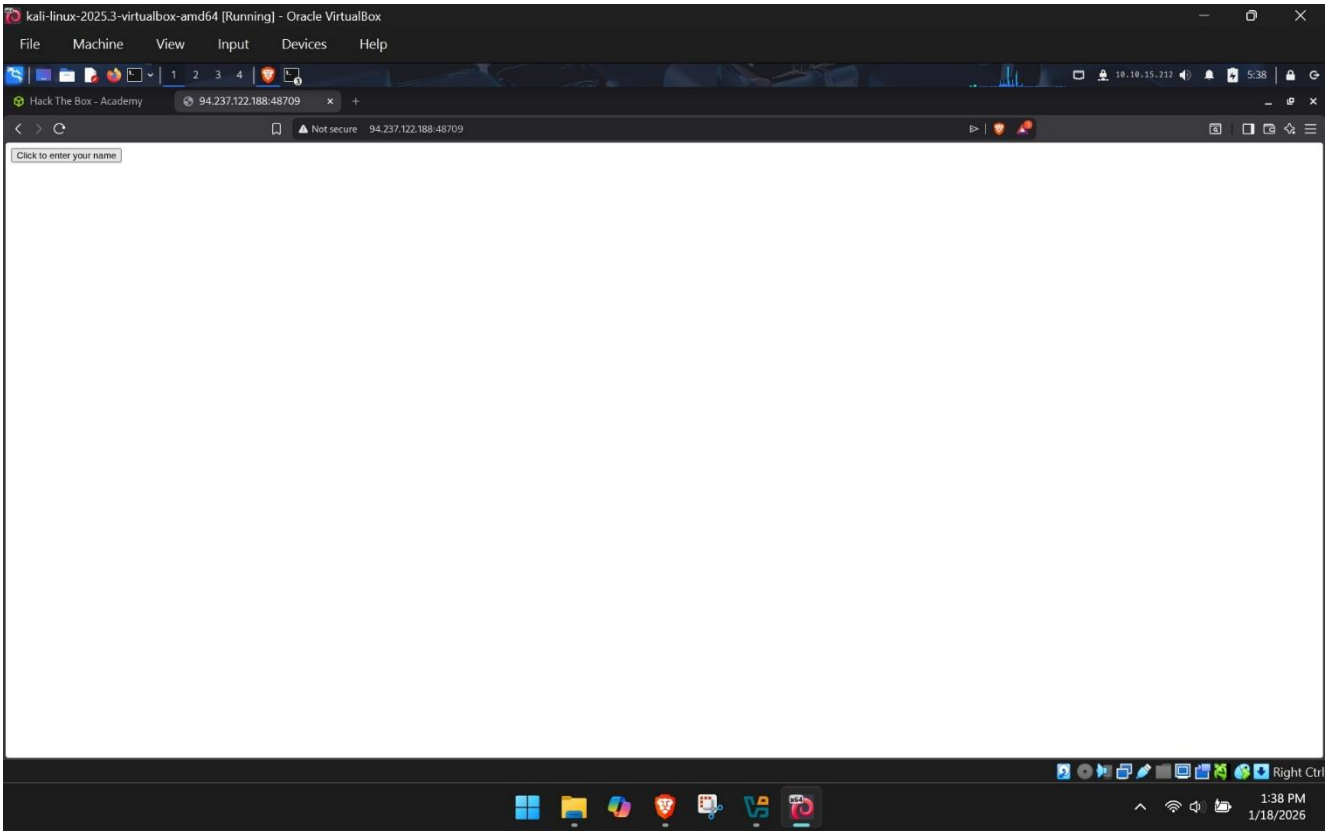


HTML Injection

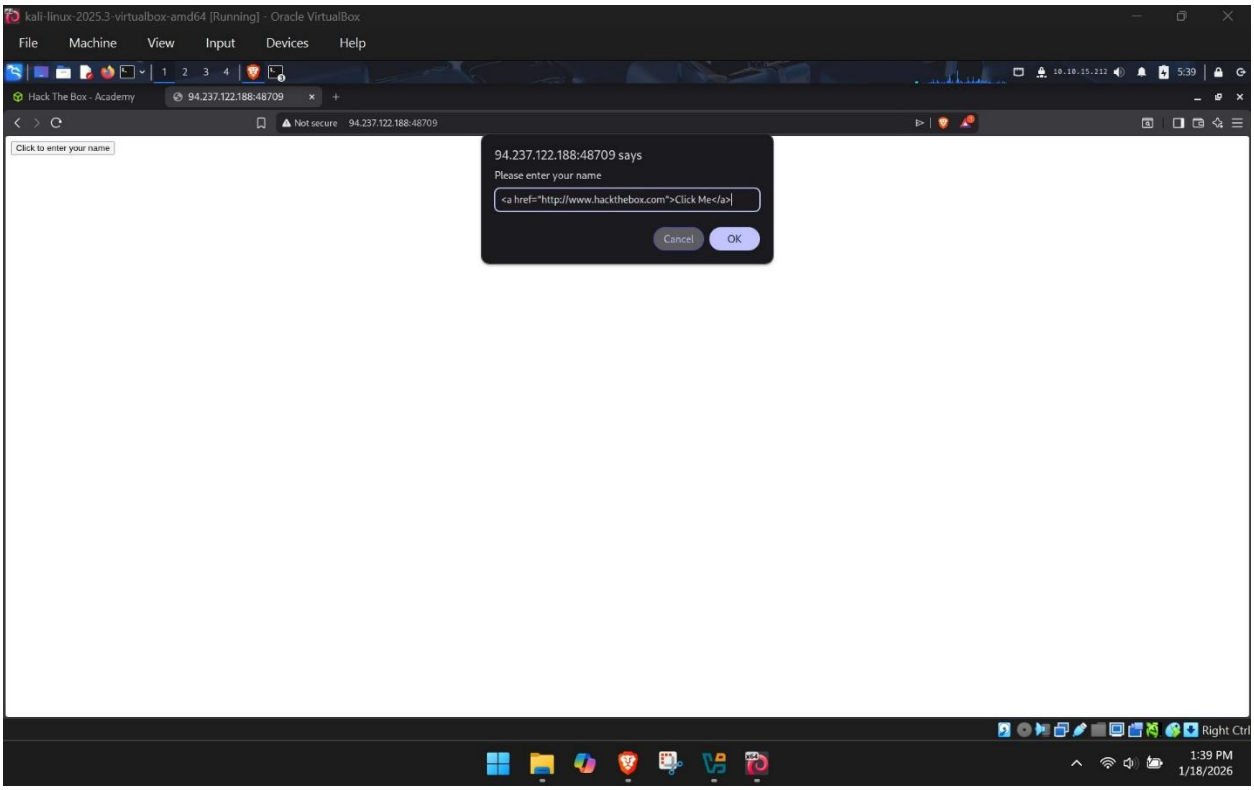
I discovered that HTML injection occurs when a webpage shows unfiltered user input, regardless of whether it comes from a database or is directly handled through JavaScript. I realized that when a user has control over the way their input looks, they can insert harmful HTML code—such as counterfeit login forms—to deceive users into giving their passwords to the attacker's server.

The question in this section illustrated a HTML injection, where I was supposed to put a malicious HTML code (payload) as an input to the target. This is shown below:

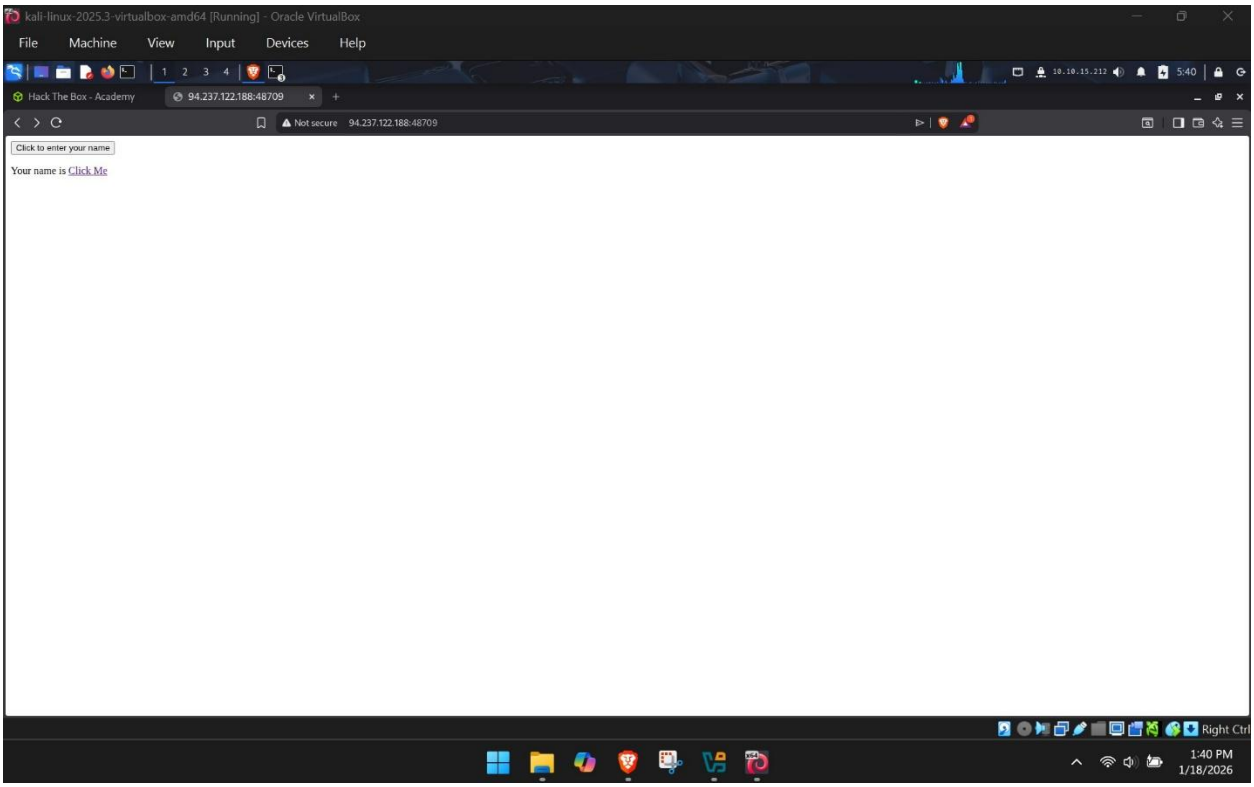

kali-linux-2025.3-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Hack The Box - Academy
94.237.122.188:48709
academy.hackthebox.com/module/75/section/757
Success
Congratulations! You earned 1 cubes!
Waiting to start...
Enable step-by-step solutions for all questions
Questions
Answer the question(s) below to complete this Section and earn cubes!
Target(s): 94.237.122.188:48709
Life Left: 88 minute(s)
+1 What text would be displayed on the page if we use the following payload as our input: <a href="http://www.hackthebox.com">Click Me</a>
Your name is Click Me
Submit
Hint
Previous
Next
+10 Streak pts
Mark Complete & Next
Powered by HACKTHEBOX
Right Ctrl
1:40 PM
1/18/2026


kali-linux-2025.3-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Hack The Box - Academy
94.237.122.188:48709
Not secure 94.237.122.188:48709
Click to enter your name
Right Ctrl
1:38 PM
1/18/2026

After loading the target shown above, I clicked the ‘Click to enter your name’ option. I then input the payload as shown below:



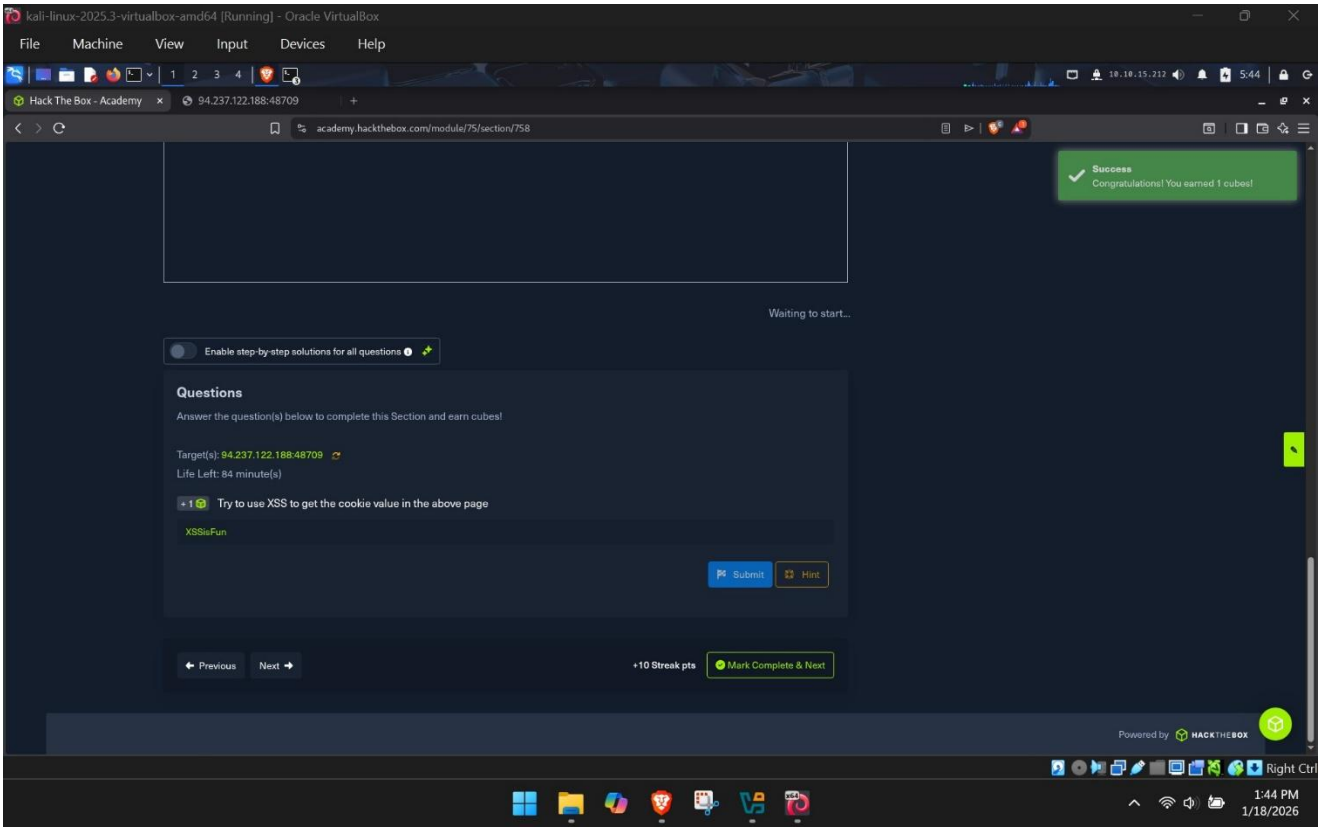
After clicking OK, the output is as shown below:



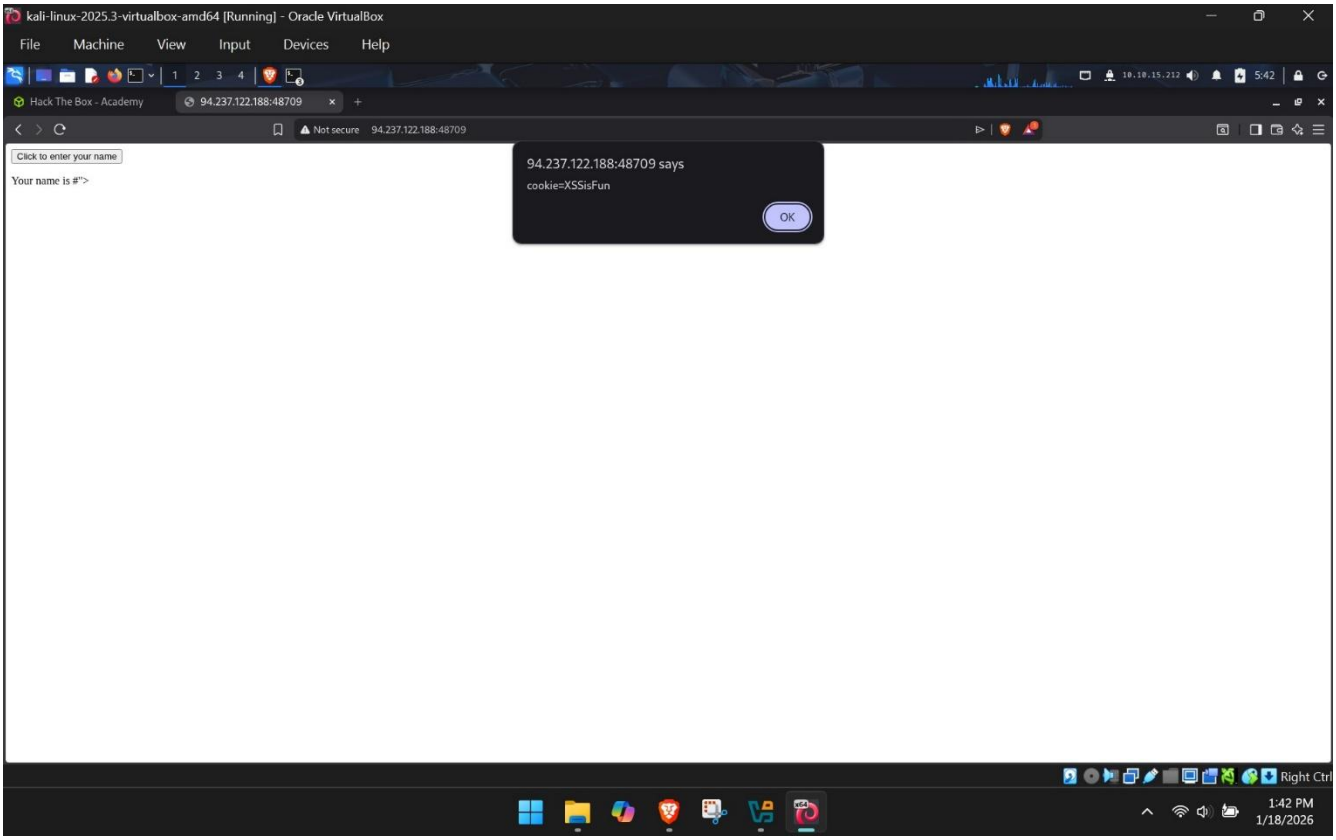
Cross-Site Scripting (XSS)

It was a fascinating journey to discover that HTML injection is often a precursor to Cross-Site Scripting (XSS) provided the attacker uses JavaScript as the injected code rather than just HTML. Security-wise, XSS could be equated to code execution on the victim's machine which allows to steal whole user accounts, whereas these two techniques might be considered alike in their working methods. Moreover, I became aware of the fact that XSS is usually classified into three main types: Reflected, Stored, and DOM-based.

The question in this section involved executing XSS to get a cookie value in a target system. This is shown below:



After inputting the malicious Javascript code as input in the target system, the cookie value was obtained as shown below:



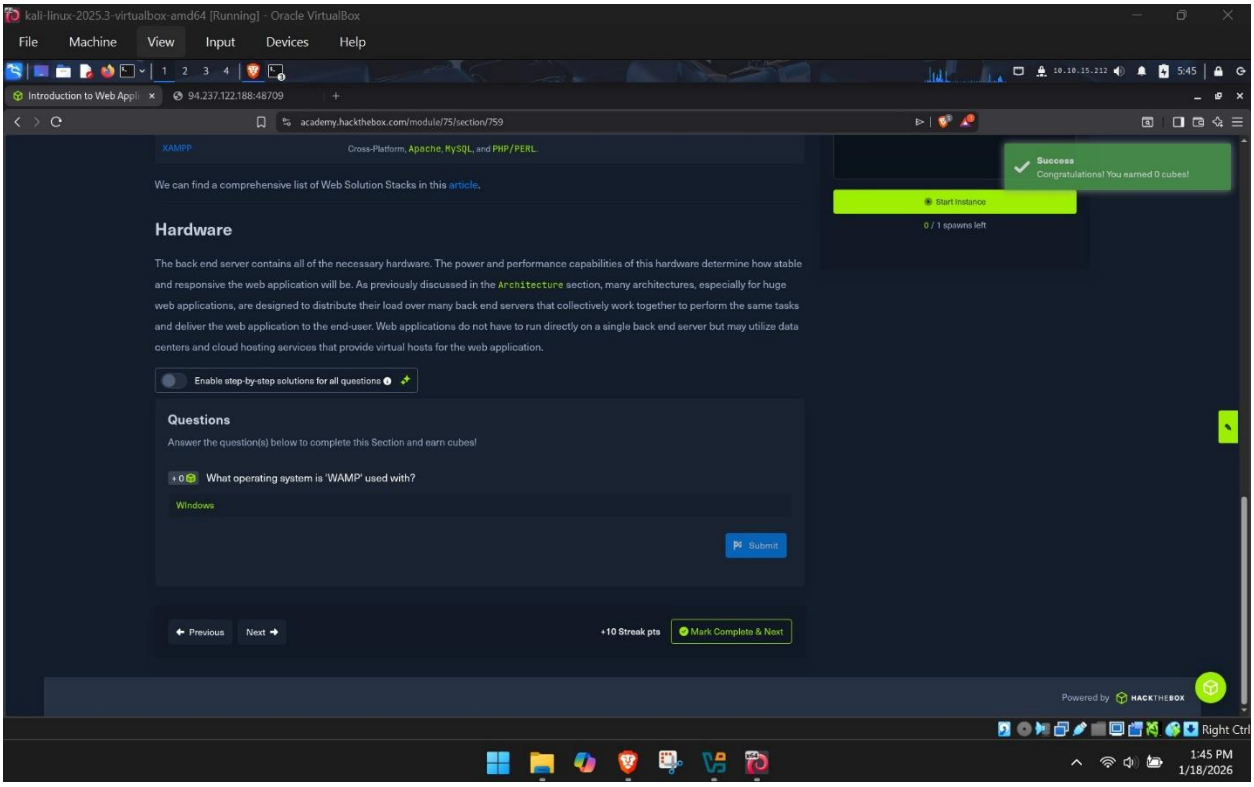
Cross-Site-Request-Forgery (CSRF)

I found out that Cross-Site Request Forgery (CSRF) can be utilized by an attacker in such a way that a victim's browser inadvertently executes unauthorized operations on an already logged-in site. I understood that these attacks usually rely on XSS vulnerabilities or use HTTP parameters manipulation to get API calls and queries done, thereby allowing the attacker to take over the user's active session and do actions pretending to be the user.

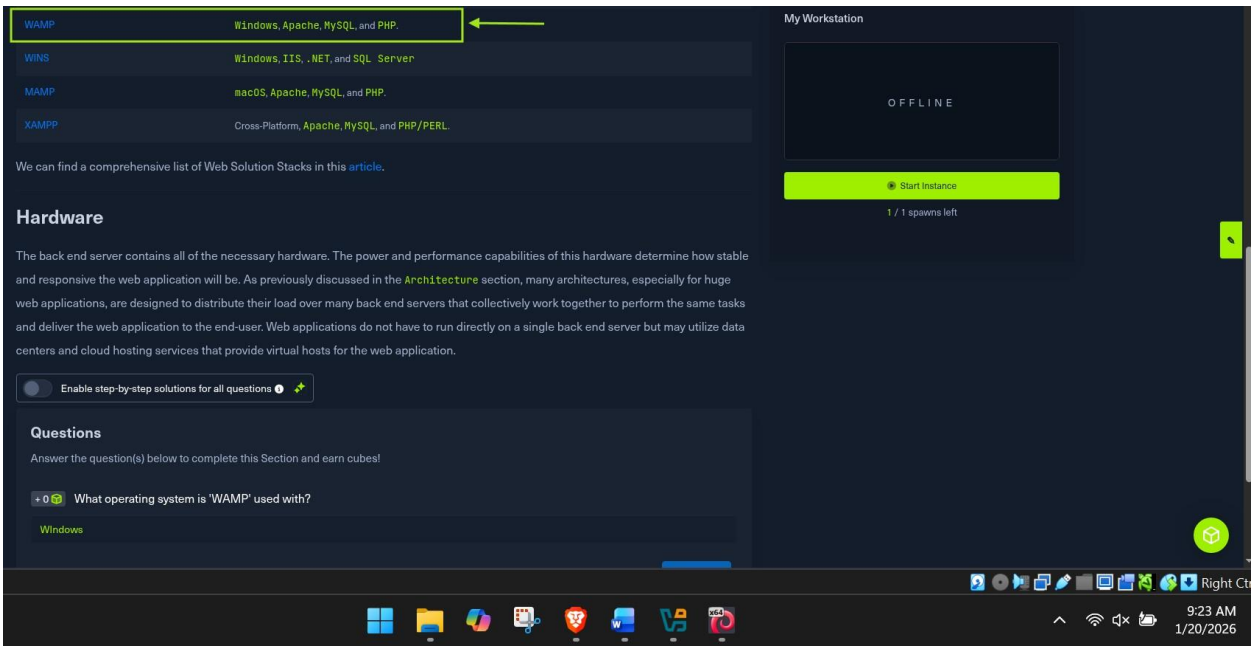
Back End Servers

I learned that the back-end server is the actual hardware and operating system which constitutes the base for the web application. I have also understood that it operates as the "engine room" where all the execution of processes and carrying out of tasks are done, and eventually connecting to the Data Access Layer of the application's architecture.

The question in this section inquired on what operating system WAMP is used in, of which the answer is shown below:



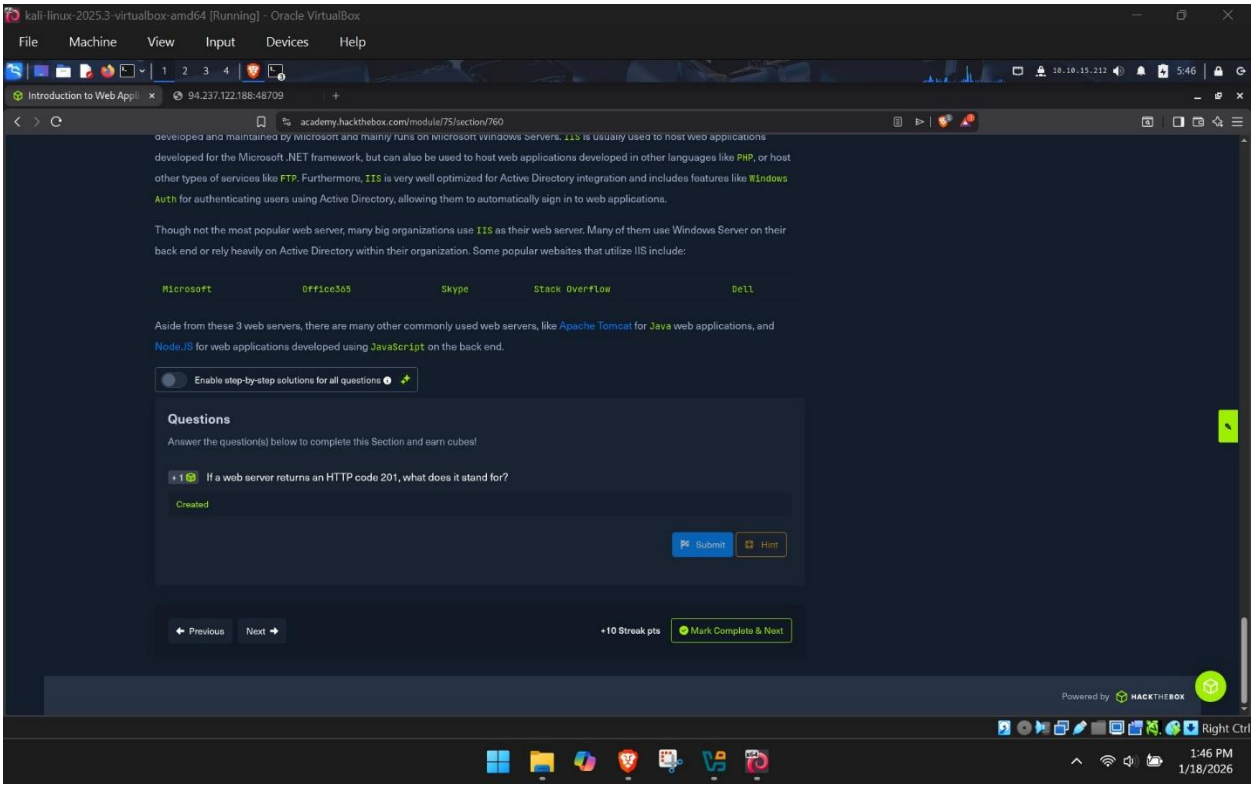
WAMP is the ‘stack’ that is used with the Windows operating system.



Web Servers

A web server is the particular software application that is active on the back-end server and controls the entire HTTP stream. I found out that its function is to get the requests from the web browser, send them to the correct parts of the website, and returns the answer. Normally, it works on port 80 or 443 and, quite practically, is the link connecting the user to all parts of the web application.

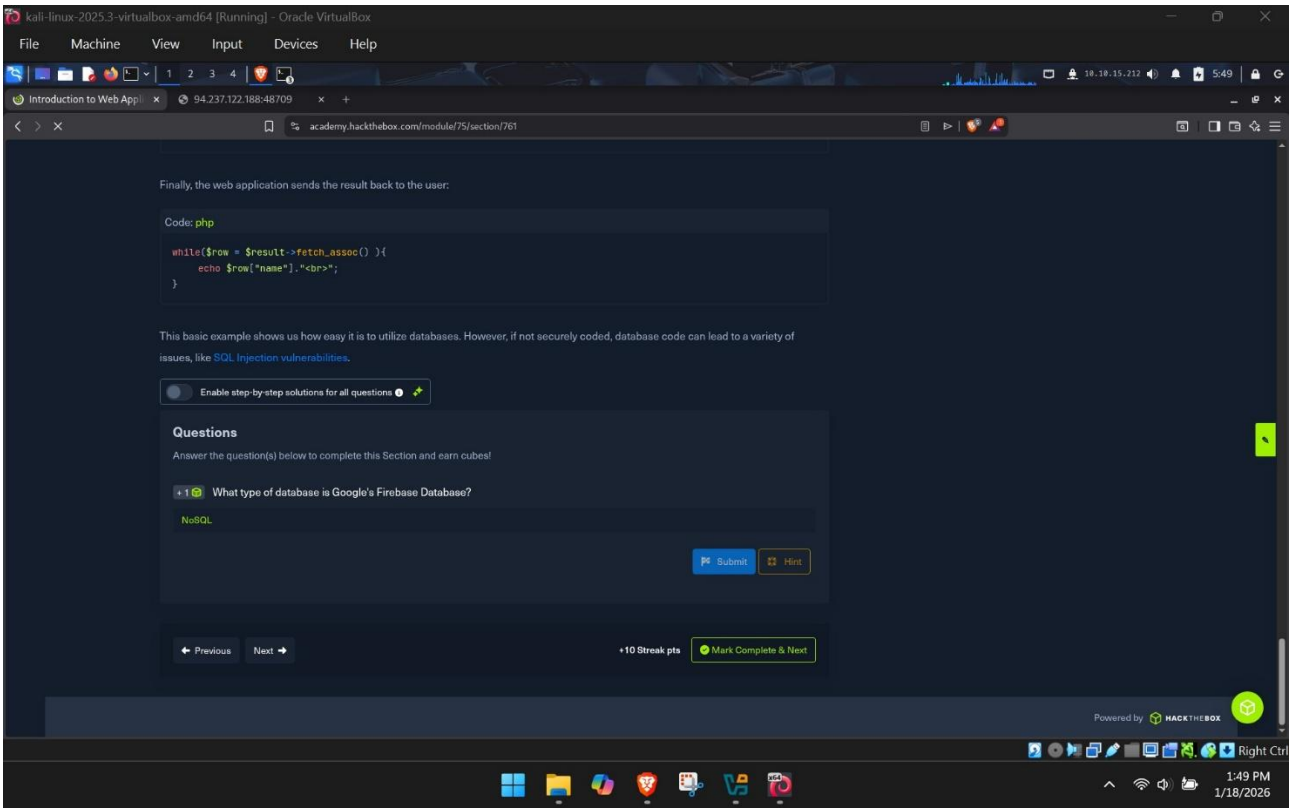
The question in this section is shown below:



Databases

I came to know that back-end databases are the storage centres for all the web application-related things. I mean, these databases contain everything regarding the web application ranging from the main assets like images to user's credentials and even the posts that are dynamically created. These databases provide each user with a personalized experience, for they let the system store and fetch data without delay. Database selection is all about finding the right mix of performance, storage size, scalability, and price that suit the application requirements.

The question in this section is shown below:

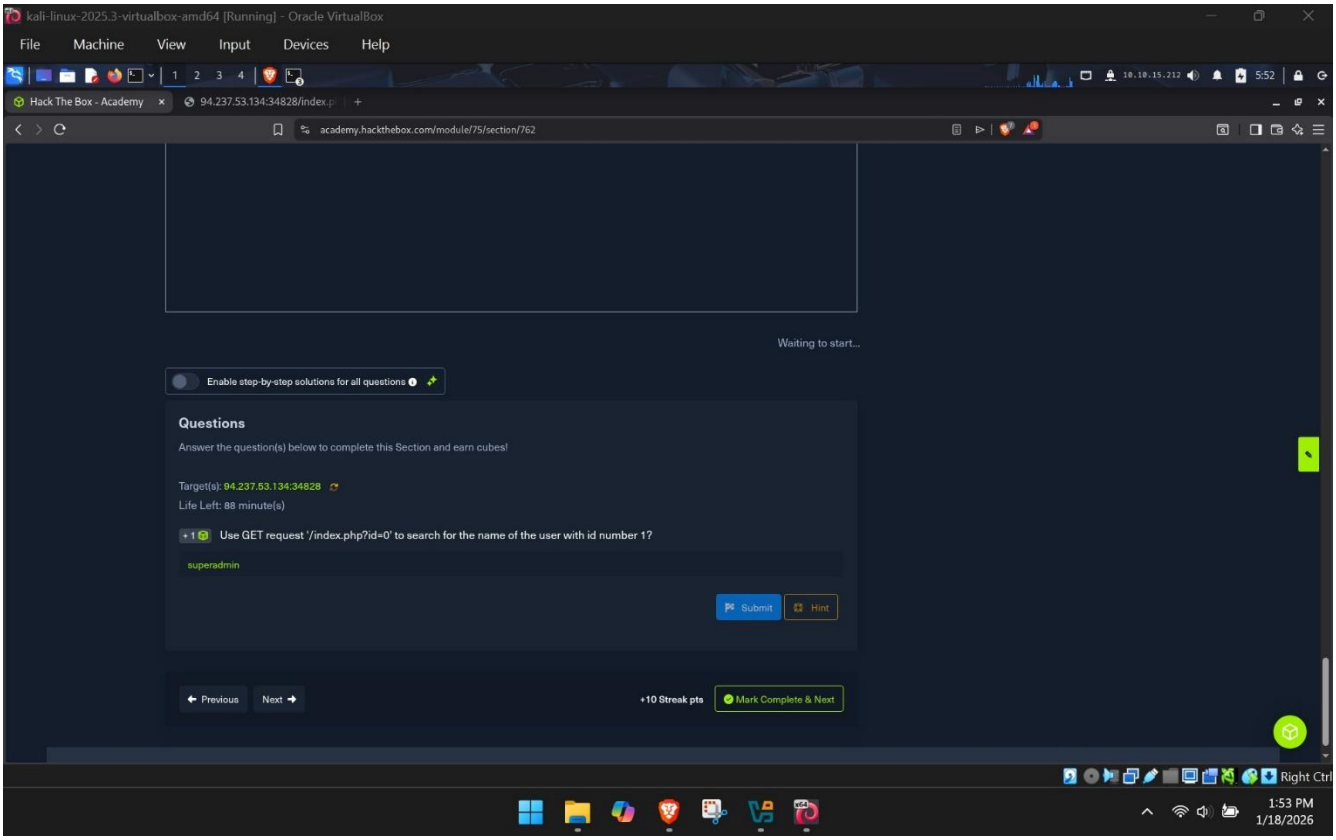


Firebase database is regarded as a NoSQL database since it utilizes non-tabular models, offers feature schemas and prioritize horizontal scalability and real-time performance as compared to the rigid structure of SQL databases.

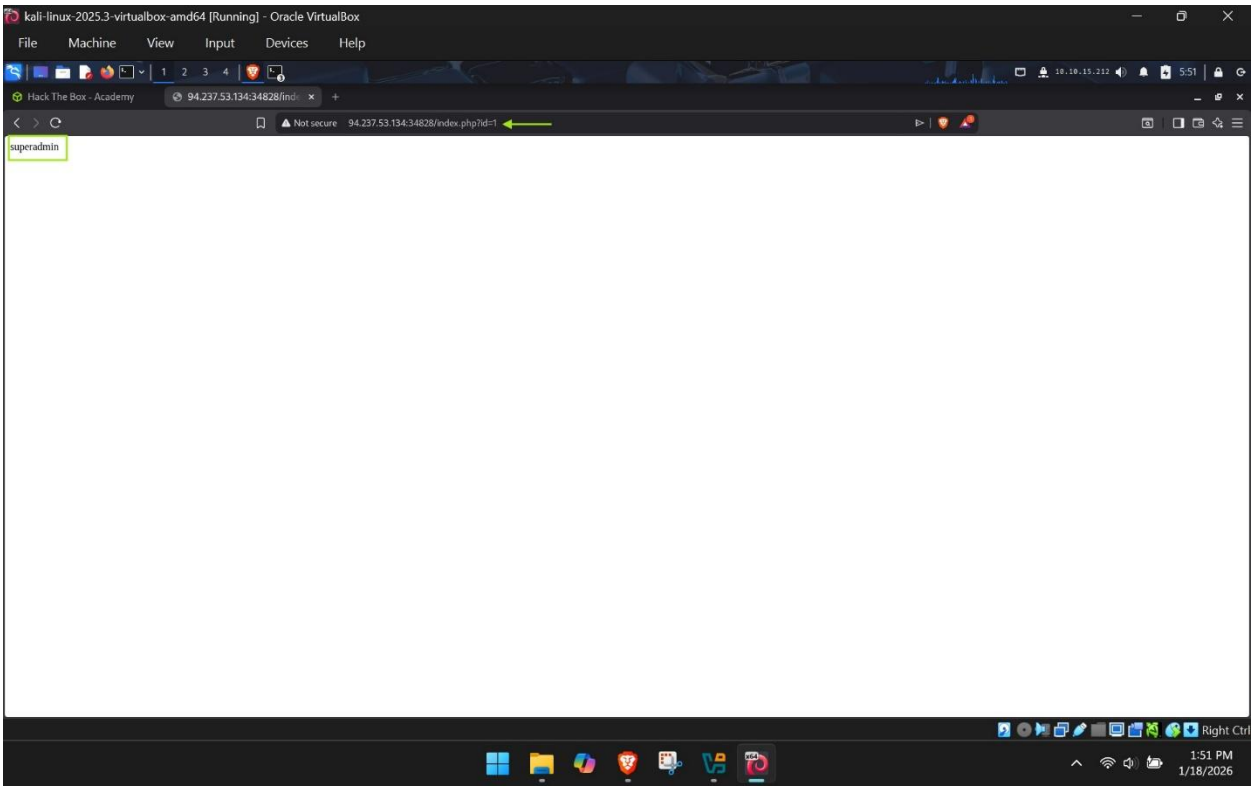
Development Frameworks and APIs

I learned that present-day web apps have their complexity managed by way of frameworks and, in addition, they make use of APIs for connecting the front and back ends. One thing I found out was when the user works with the front end, it makes a request through an API to the back end, which then does the data processing and returns the response to the browser for the user to see.

Question



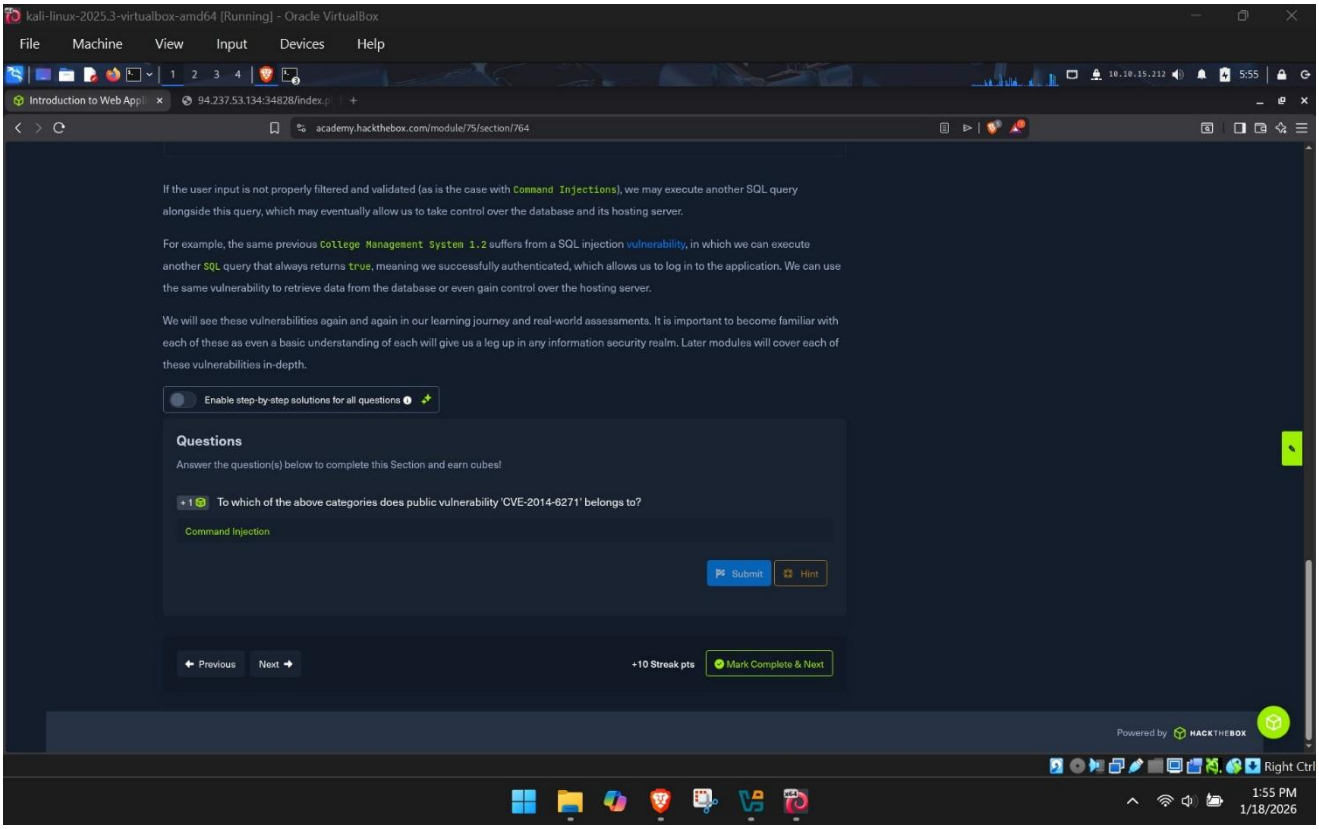
In order to get the name of the user with id number = 1, I had to input an additional part in the URL, which was ***'index.php?id=1'.*** The output is shown below:



Common Web Vulnerabilities

I discovered that when public exploits are absent, One can find themselves relying on manual techniques to search for vulnerabilities or to find weaknesses created by developers' misconfigurations. I got to know that wrong setups can lead to security breaches even in secure applications and that the majority of these everyday problems fall within the OWASP Top 10, which is an acknowledgment of the most severe web application security risks.

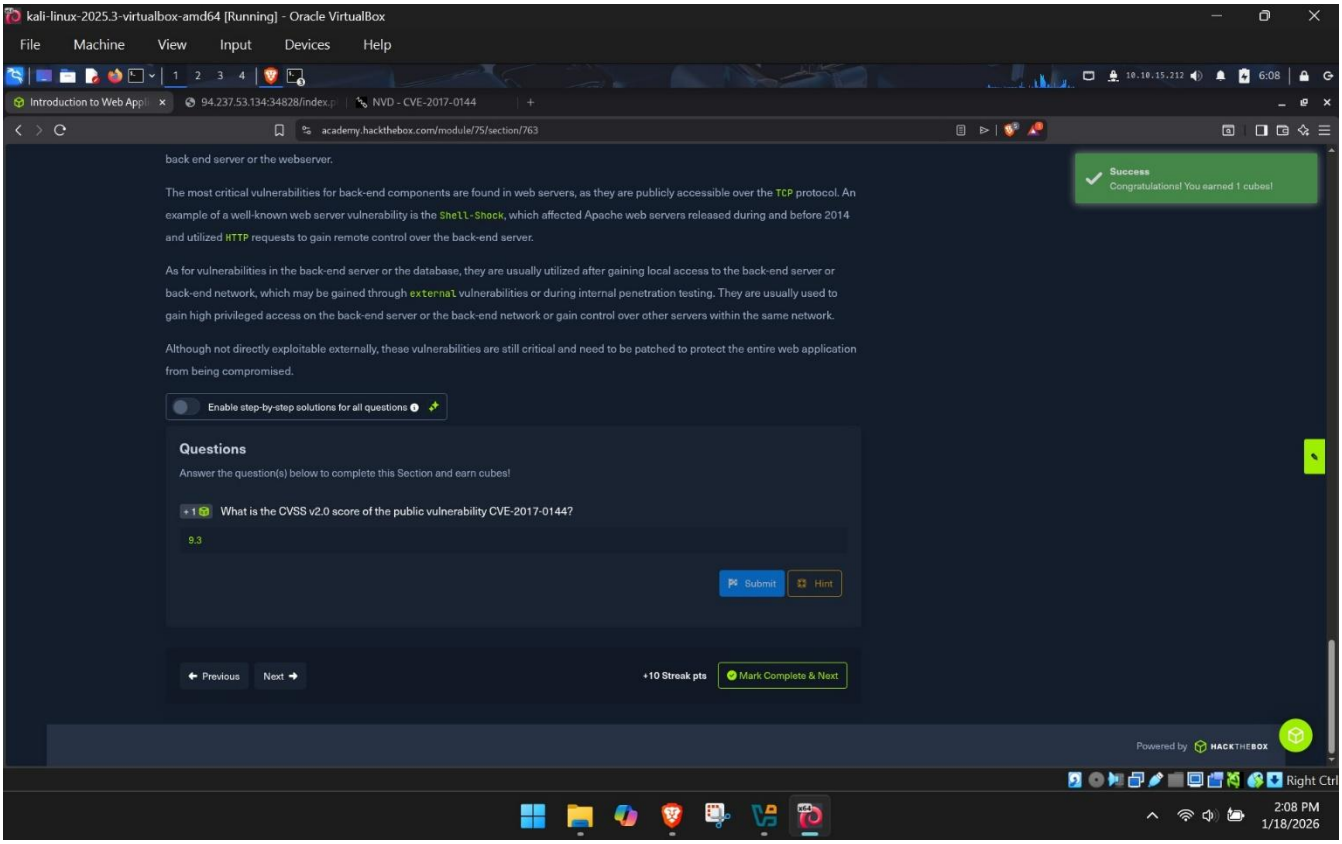
Question



Public Vulnerabilities

Back-end vulnerabilities are the result of coding errors that are able to be exploited by outsiders taking over a server completely via remote access, and these mistakenly written codes are still the most critical vulnerabilities behind the back end. Such a situation led me to think that since the web applications that are oftentimes used by many people are under constant scrutiny by the professionals who are spread all over the planet, the vulnerabilities that come along with them would be often found, fixed, and recorded in the form of Common Vulnerabilities and Exposures (CVEs). Besides, I learned that organizations rely on the Common Vulnerability Scoring System (CVSS) standard to assess these threats, which in turn facilitates them to allocate their security resources according to the priority order of the risks with the least and then the most dangerous risks being responded to first.

Question



The CVSS v2.0 score of the vulnerability CVE-2017-0144 is 9.3, which is rated as high. I obtained the value by searching in a website with this information. In my case, I utilized the nvd.nist.gov website to check the score, which is shown below:

File Machine View Input Devices Help

Introduction to Web Applicati | 94.237.53.134:34828/index.p | NVD - CVE-2017-0144

nvd.nist.gov/vuln/detail/CVE-2017-0144

CVE-2017-0144 Detail

DEFERRED

This CVE record is not being prioritized for NVD enrichment efforts due to resource or other concerns.

Description

The SMBv1 server in Microsoft Windows Vista SP2; Windows Server 2008 SP2 and R2 SP1; Windows 7 SP1; Windows 8.1; Windows Server 2012 Gold and R2; Windows RT 8.1; and Windows 10 Gold, 1511, and 1607; and Windows Server 2016 allows remote attackers to execute arbitrary code via crafted packets, aka "Windows SMB Remote Code Execution Vulnerability." This vulnerability is different from those described in CVE-2017-0143, CVE-2017-0145, CVE-2017-0146, and CVE-2017-0148.

QUICK INFO

CVE Dictionary Entry:
CVE-2017-0144
NVD Published Date:
03/16/2017
NVD Last Modified:
10/21/2025
Source:
Microsoft Corporation

Metrics

CVSS Version 4.0 | CVSS Version 3.x | CVSS Version 2.0

NVD enrichment efforts reference publicly available information to associate vector strings. CVSS information contributed by other sources is also displayed.

CVSS 2.0 Severity and Vector Strings:

NIST: NVD **Base Score:** 9.3 HIGH **Vector:** (AV:N/AC:M/Au:N/C:C/I:C/A:C)

References to Advisories, Solutions, and Tools

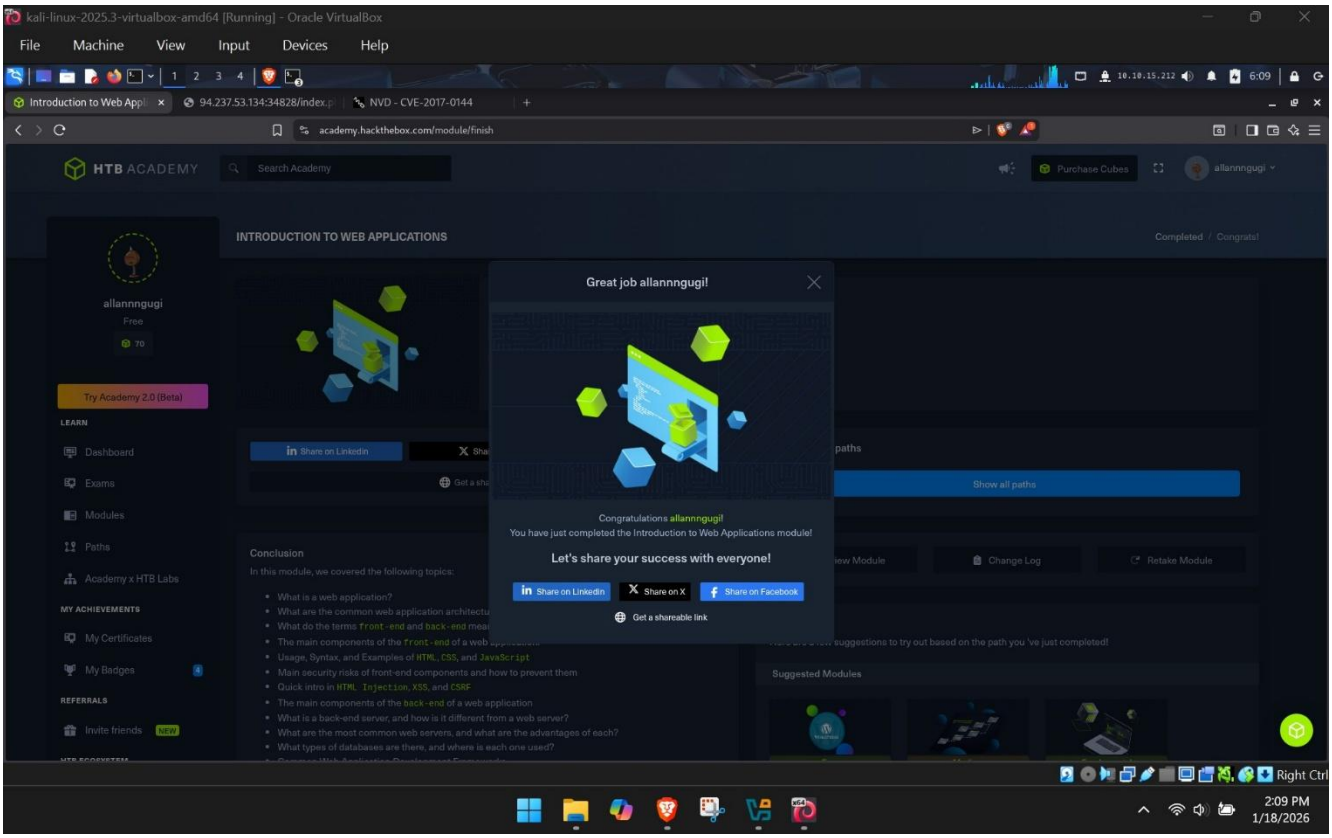
By selecting these links, you will be leaving NIST webspace. We have provided these links to other web sites because they may have information that would be of interest to you. No inferences should be drawn on account of other sites being referenced, or not, from this page. There may be other web sites that are more appropriate for your purpose. NIST does not necessarily endorse the views expressed, or concur with the facts presented on these sites. Further, NIST does not endorse any commercial products that may be mentioned on these sites. Please address comments about this page to nvd@nist.gov.

URL	Source(s)	Tag(s)
http://packetstormsecurity.com/files/154690/DOUBLEPULSAR-Payload-Execution-Neutralization.html	CVE, Microsoft Corporation	Exploit, Third Party Advisory, VDB Entry
http://packetstormsecurity.com/files/156196/SMB-DOUBLEPULSAR-Remote-Code-	CVE, Microsoft	Exploit, Third Party Advisory

Right Ctrl

2:02 PM
1/18/2026

Module Completion



Conclusion

Completing the "Introduction to Web Applications" module on Hack The Box not only gave me a solid foundation in web technologies but also in vulnerability exploitation. The course introduced me to the concepts of HTTP, to the interactions between client and server, and to the conditions that lead to the security flaws of SQL injection and XSS. Besides being able to identify vulnerabilities and understand the attacker's mindset perfectly, the hands-on labs greatly increased my confidence in these aspects. I am now looking at the whole picture from the security perspective and have developed a very positive attitude toward proactive defence and secure coding.